

要走多长的距离。不难发现 $d(i,j)=d(j,i)$ ，因此从现在开始规定在状态中 $i>j$ 。这样，不管是哪个人，下一步只能走到 $i+1, i+2, \dots$ 这些点。可是，如果走到 $i+2$ ，情况变成了“1~ i 和 $i+2$ ，但是 $i+1$ 没走过”，无法表示成状态！怎么办？禁止这样的决策！也就是说，只允许其中一个人走到 $i+1$ ，而不能走到 $i+2, i+3, \dots$ 。换句话说，状态 $d(i,j)$ 只能转移到 $d(i+1,j)$ 和 $d(i+1,i)$ ^①。

可是这样做产生了一个问题：上述“霸道”的规定是否可能导致漏解呢？不会。因为如果第一个人直接走到了 $i+2$ ，那么它再也无法走到 $i+1$ 了，只能靠第二个人走到 $i+1$ 。既然如此，现在就让第二个人走到 $i+1$ ，并不会丢失解。

边界是 $d(n-1,j)=\text{dist}(n-1,n)+\text{dist}(j,n)$ ，其中 $\text{dist}(a,b)$ 表示点 a 和 b 之间的距离。因为根据定义，所有点都走过了，两个人只需直接走到终点。所求结果是 $\text{dist}(1,2)+d(2,1)$ ，因为第一步一定是某个人走到了第二个点，根据定义，这就是 $d(2,1)$ 。

状态总数有 $O(n^2)$ 个，每个状态的决策只有两个，因此总时间复杂度为 $O(n^2)$ 。

9.3 多阶段决策问题

还记得“多阶段决策问题”吗？在回溯法中曾提到过该问题。简单地说，每做一次决策就可以得到解的一部分，当所有决策做完之后，完整的解就“浮出水面”了。在回溯法中，每次决策对应于给一个结点产生新的子树，而解的生成过程对应一棵解答树，结点的层数就是“下一个待填充位置” cur 。

9.3.1 多段图的最短路

多段图是一种特殊的 DAG，其结点可以划分成若干个阶段，每个阶段只由上一个阶段所决定。下面举一个例子：

例题 9-4 单向 TSP (Unidirectional TSP, UVa 116)

给一个 m 行 n 列 ($m \leq 10, n \leq 100$) 的整数矩阵，从第一列任何一个位置出发每次往右、右上或右下走一格，最终到达最后一列。要求经过的整数之和最小。整个矩阵是环形的，即第一行的上一行是最后一行，最后一行的下一行是第一行。输出路径上每列的行号。多解时输出字典序最小的。图 9-5 中是两个矩阵和对应的最优路线（唯一的区别是最后一行）。

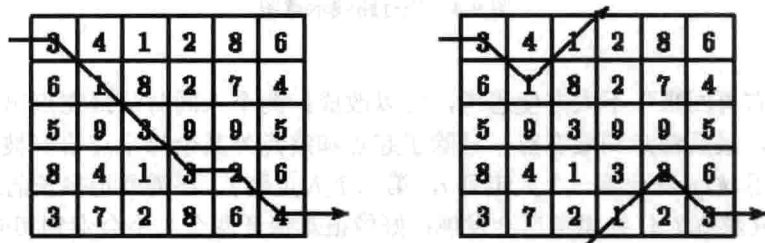


图 9-5 矩阵对应的最优路线

^① 第二个人走到 $i+1$ 时本应转移到 $d(i,i+1)$ ，但是根据此处规定，必须写成 $d(i+1,i)$ 。

【分析】

在这个题目中，每一列就是一个阶段，每个阶段都有3种决策：直行、右上和右下。

提示 9-13：多阶段决策的最优化问题往往可以用动态规划解决，其中，状态及其转移类似于回溯法中的解答树。解答树中的“层数”，也就是递归函数中的“当前填充位置”*cur*，描述的是即将完成的决策序号，在动态规划中被称为“阶段”。

有了前面的经验，不难设计出状态：设 $d(i,j)$ 为从格子 (i,j) 出发到最后一列的最小开销。但是本题不仅要输出解，还要求字典序最小，这就需要在计算 $d(i,j)$ 的同时记录“下一列的行号”的最小值（当然是在满足最优性的前提下），细节参见代码：

```
int ans = INF, first = 0;
for(int j = n-1; j >= 0; j--) { //逆推
    for(int i = 0; i < m; i++) {
        if(j == n-1) d[i][j] = a[i][j]; //边界
        else {
            int rows[3] = {i, i-1, i+1};
            if(i == 0) rows[1] = m-1; //第0行“上面”是第m-1行
            if(i == m-1) rows[2] = 0; //第m-1行“下面”是第0行
            sort(rows, rows+3); //重新排序，以便找到字典序最小的
            d[i][j] = INF;
            for(int k = 0; k < 3; k++) {
                int v = d[rows[k]][j+1] + a[i][j];
                if(v < d[i][j]) { d[i][j] = v; next[i][j] = rows[k]; }
            }
            if(j == 0 && d[i][j] < ans) { ans = d[i][j]; first = i; }
        }
    }
    printf("%d", first+1); //输出第1列
    for(int i = next[first][0], j = 1; j < n; i = next[i][j], j++)
        printf(" %d", i+1); //输出其他列
    printf("\n%d\n", ans);
}
return 0;
}
```

9.3.2 0-1 背包问题

0-1 背包问题是最广为人知的动态规划问题之一，拥有很多变形。尽管在理解之后并不难写出程序，但初学者往往需要较多的时间才能掌握它。在介绍 0-1 背包问题之前，先来看一个引例。

物品无限的背包问题。有 n 种物品，每种均有无穷多个。第 i 种物品的体积为 V_i ，重量

为 W_i 。选一些物品装到一个容量为 C 的背包中，使得背包内物品在总体积不超过 C 的前提下重量尽量大。 $1 \leq n \leq 100$, $1 \leq V_i \leq C \leq 10000$, $1 \leq W_i \leq 10^6$ 。

【分析】

很眼熟是吗？没错，它很像 9.2 节中的硬币问题，只不过“面值之和恰好为 S ”改成了“体积之和不超过 C ”，另外增加了一个新的属性——重量，相当于把原来的无权图改成了带权图（weighted graph）。这样，问题就变为了求以 C 为起点（终点任意）的、边权之和最大的路径。

与前面相比，DAG 从“无权”变成了“带权”，但这并没有带来任何困难，此时只需将某处代码从“+1”变成“+W[i]”即可。你能找到吗？

提示 9-14：动态规划的适用性很广。不少可以用动态规划解决的题目，在条件稍微变化后只需对状态转移方程做少量修改即可解决新问题。

0-1 背包问题。有 n 种物品，每种只有一个。第 i 种物品的体积为 V_i ，重量为 W_i 。选一些物品装到一个容量为 C 的背包，使得背包内物品在总体积不超过 C 的前提下重量尽量大。 $1 \leq n \leq 100$, $1 \leq V_i \leq C \leq 10000$, $1 \leq W_i \leq 10^6$ 。

【分析】

不知读者有没有发现，刚才的方法已经不适用了：只凭“剩余体积”这个状态，无法得知每个物品是否已经用过。换句话说，原来的状态转移太乱了，任何时候都允许使用任何一种物品，难以控制。为了消除这种混乱，需要让状态转移（也就是决策）有序化。

引入“阶段”之后，算法便不难设计了：用 $d(i, j)$ 表示当前在第 i 层，背包剩余容量为 j 时接下来的最大重量和，则 $d(i, j) = \max\{d(i+1, j), d(i+1, j - V[i]) + W[i]\}$ ，边界是 $i > n$ 时 $d(i, j) = 0$, $j < 0$ 时为负无穷（一般不会初始化这个边界，而是只当 $j \geq V[i]$ 时才计算第二项）。

说得更通俗一点， $d(i, j)$ 表示“把第 $i, i+1, i+2, \dots, n$ 个物品装到容量为 j 的背包中的最大总重量”。事实上，这个说法更加常用——“阶段”只是辅助思考的，在动态规划的状态描述中最好避免“阶段”、“层”这样的术语。很多教材和资料直接给出了这样的状态描述，而本书中则是花费了大量的篇幅叙述为什么会想到要划分阶段以及和回溯法的内在联系——如果对此理解不够深入，很容易出现“每次碰到新题自己都想不出来，但一看题解就懂”的尴尬情况。

提示 9-15：学习动态规划的题解，除了要理解状态表示及其转移方程外，最好思考一下为什么会想到这样的状态表示。

和往常一样，在得到状态转移方程之后，还需思考如何编写程序。尽管在很多情况下，记忆化搜索程序更直观、易懂，但在 0-1 背包问题中，递推法更加理想。为什么呢？因为有了“阶段”定义后，计算顺序变得非常明显。

提示 9-16：在多阶段决策问题中，阶段定义了天然的计算顺序。

下面是代码，答案是 $d[1][C]$ ：

```
for(int i = n; i >= 1; i--)
    for(int j = 0; j <= C; j++){
```

```

d[i][j] = (i==n ? 0 : d[i+1][j]);
if (j >= V[i]) d[i][j] = max(d[i][j], d[i+1][j-V[i]]+W[i]);
}

```

前面说过, i 必须逆序枚举, 但 j 的循环次序是无关紧要的。

规划方向。聪明的读者也许看出来了, 还有另外一种“对称”的状态定义: 用 $f(i, j)$ 表示“把前 i 个物品装到容量为 j 的背包中的最大总重量”, 其状态转移方程也不难得出:

$$f(i, j) = \max\{f(i-1, j), f(i-1, j-V[i]) + W[i]\}$$

边界是类似的: $i=0$ 时为 0, $j<0$ 时为负无穷, 最终答案为 $f(n, C)$ 。代码也是类似的:

```

for(int i = 1; i <= n; i++){
    for(int j = 0; j <= C; j++){
        f[i][j] = (i==1 ? 0 : f[i-1][j]);
        if(j >= V[i]) f[i][j] = max(f[i][j], f[i-1][j-V[i]]+W[i]);
    }
}

```

看上去这两种方式是完全对称的, 但其实存在细微区别: 新的状态定义 $f(i, j)$ 允许边读入边计算, 而不必把 V 和 W 保存下来。

```

for(int i = 1; i <= n; i++){
    scanf("%d%d", &V, &W);
    for(int j = 0; j <= C; j++){
        f[i][j] = (i==1 ? 0 : f[i-1][j]);
        if(j >= V) f[i][j] = max(f[i][j], f[i-1][j-V]+W);
    }
}

```

滚动数组。更奇妙的是, 还可以把数组 f 变成一维的:

```

memset(f, 0, sizeof(f));
for(int i = 1; i <= n; i++){
    scanf("%d%d", &V, &W);
    for(int j = C; j >= 0; j--){
        if(j >= V) f[j] = max(f[j], f[j-V]+W);
    }
}

```

为什么这样做是正确的呢? 下面来看一下 $f(i, j)$ 的计算过程, 如图 9-6 所示。

f 数组是从上到下、从右往左计算的。在计算 $f(i, j)$ 之前, $f[j]$ 里保存的就是 $f(i-1, j)$ 的值, 而 $f[j-W]$ 里保存的是 $f(i-1, j-W)$ 而不是 $f(i, j-W)$ ——别忘了 j 是逆序枚举的, 此时 $f(i, j-W)$ 还没有算出来。这样, $f[j] = (\max\{f[j], f[j-V]+W\})$ 实际上是把 $\max\{f(i-1, j), f(i-1, j-V)\}$ 保存在 $f[j]$ 中, 覆盖掉 $f[j]$ 原来的 $f(i-1, j)$ 。

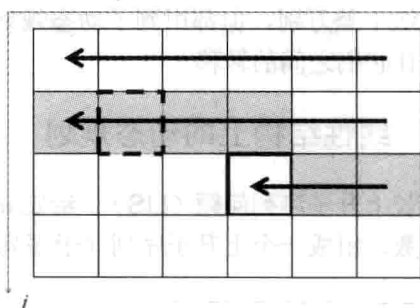


图 9-6 0-1 背包问题的计算顺序